训练服务器指导手册

1、服务器登录

1.1、事项说明

由于 GPU 卡资源有限，规定一个队伍只能使用一张卡进行训练，否则取消服务器的使用权限。大家可以选择空闲 GPU 卡或者 Volatile GPU-Util 利用率小于 85%的 GPU 卡进行训练，建议优先使用空闲卡进行训练；

1.2、云安全访问服务登录

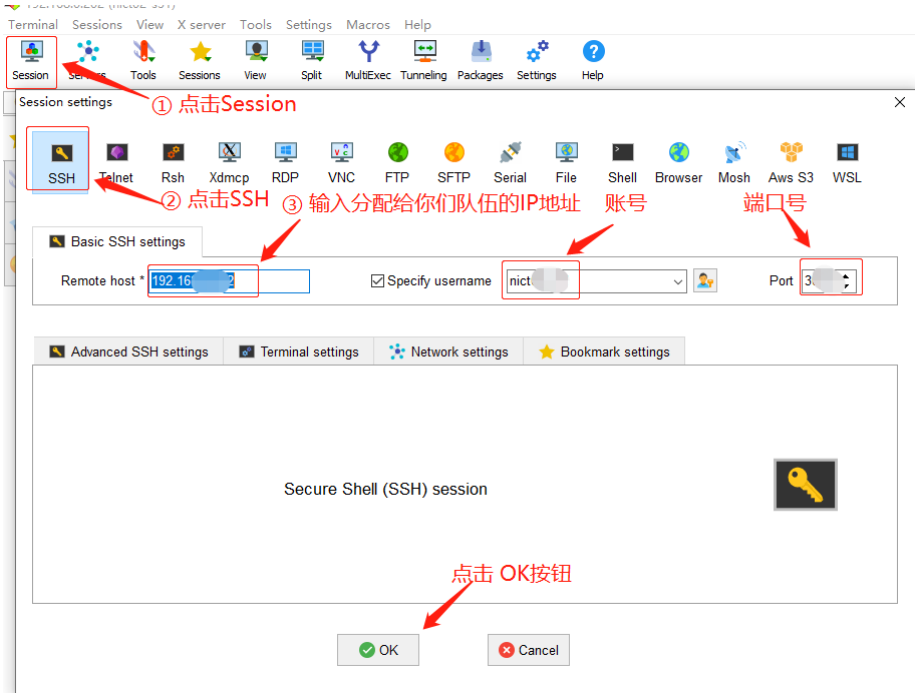
- 在访问服务器之前，需要登录云安全访问服务，首先下载一个云安全访问服务客户端，点击云安全服务器客户端下载即可。 下载安装成功后，如下图所示：

- 一个队伍对应一个 云安全访问服务账号，具体以发布为准，大家找到各自队伍对应的云安全访问服务账号和密码即可， 登录成功后，如下图所示：

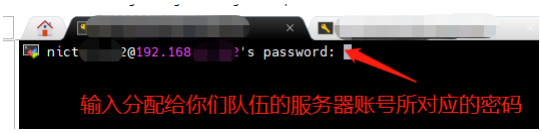
- 若需要退出云安全访问服务客户端，密码为 **Nihao123456!**
- 若需要卸载云安全访问服务客户端，密码为空

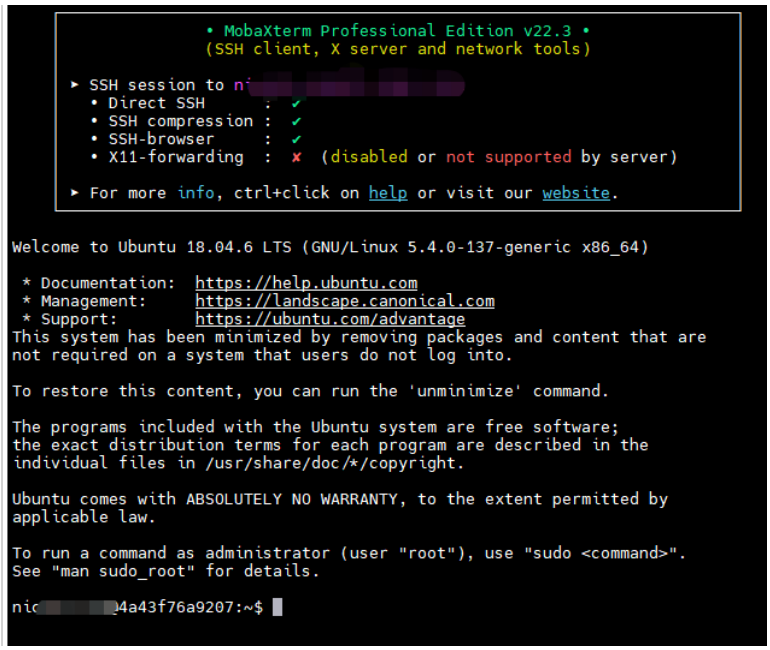
1.3、使用SSH登录训练服务器

- 步骤1：下载MobaXterm工具,当然如果你电脑里面有安装其他的ssh工具也是可以的。
- 步骤2：获取你所在队伍的服务器账号、密码、端口号等，这个可以咨询海思老师或者队长。
- 步骤3：使用MobaXterm登录到，队伍所分配的服务器的ssh。



- 步骤4：输入分配给你们队伍的服务器账号所对应的密码，然后敲回车，就能进入到训练服务器的命令行终端了。





- 步骤5：在训练服务器的命令行终端，输入下面的命令，即可看到训练服务器的CUDA版本为11.3

```
nvcc  -V
```

```
See  man sudo_root  for details.

ni      @4a43f76a9207:~$ nvcc  -V
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2021 NVIDIA Corporation
Built on Mon_May__3_19:15:13_PDT_2021
Cuda compilation tools, release 11.3, V11.3.109
Build cuda_11.3.r11.3/compiler.29920130_0
nic     @4a43f76a9207:~$
```

1.4、GPU卡使用规则和命名查看

- 用户上线后，可通过 nvidia-smi 查看当前 GPU 资源占用情况，如下图所示：
- 针对下图 nvidia-smi 参数进行解读：
  - GPU：GPU 编号； Name：GPU 型号；
  - Persistence-M：持续模式的状态。持续模式虽然耗能大，但是在新的 GPU 应用启动时，花费的时 间更少，这里显示的是 off 的状态；
  - Fan：风扇转速，从 0 到 100%之间变动；
  - Temp：温度，单位是摄氏度；
  - Perf：性能状态，从 P0 到 P12，P0 表示最大性能，P12 表示状态最小性能（即 GPU 未工作时为 P0，达到最大工作限度时为 P12）
  - Pwr:Usage/Cap：能耗；
  - Memory Usage：显存使用率；
  - Bus-Id：涉及 GPU 总线的东西，domain ： bus:device.function；
  - Disp.A：Display Active，表示 GPU 的显示是否初始化；
  - Volatile GPU-Util：浮动的 GPU 利用率；
  - Uncorr. ECC：Error Correcting Code，错误检查与纠正；
  - Compute M：compute mode，计算模式。
  - 下方的 Processes 表示每个进程对 GPU 的显存使用率。
- **注：若 Volatile GPU-Util 利用率为 0 ，则证明该显卡为空闲，表示 可以使用，或者某张卡 Volatile GPU-Util 利用率小于 85%也可以使用这张卡，建议优先使用空闲卡 进行训练。**

```
Build cuda_11.3.r11.3/compiler.29920130_0
nic     @4a43f76a9207:~$ nvidia-smi
Fri Feb 10 17:47:32 2023
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 515.48.07    Driver Version: 515.48.07    CUDA Version: 11.7     |
|-------------------------------+----------------------+----------------------+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|                               |                      |               MIG M. |
|===============================+======================+======================|
|   0  Tesla T4            Off  | 00000000:86:00.0 Off |                    0 |
| N/A   34C    P0    28W /  70W |      2MiB / 15360MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
|   1  Tesla T4            Off  | 00000000:87:00.0 Off |                    0 |
| N/A   38C    P0    27W /  70W |      2MiB / 15360MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
|   2  Tesla T4            Off  | 00000000:AF:00.0 Off |                    0 |
| N/A   36C    P0    27W /  70W |      2MiB / 15360MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
|   3  Tesla T4            Off  | 00000000:D8:00.0 Off |                    0 |
| N/A   38C    P0    28W /  70W |      2MiB / 15360MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------+
| Processes:                                                                  |
|  GPU   GI   CI        PID   Type   Process name                  GPU Memory |
|        ID   ID                                                   Usage      |
|=============================================================================|
|  No running processes found                                                 |
+-----------------------------------------------------------------------------+
```

海思目前只提供了基于Resnet18的分类网和YOLOV2的检测网，如果开发者的参赛作品需要用到其他网络的话，需要自己进行训练环境搭建、模型训练、模型转换等工作。

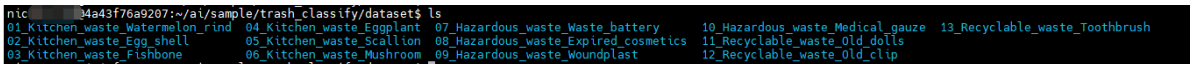
2、分类网模型训练及模型转换

步骤1：数据集准备

- 在进行分类网训练之前，请您仔细阅读社区提供的分类网训练前的准备工作，如：数据集的制作、数据集的清洗、数据集的标注等，具体请参考《5.3.2.3分类网（resent）》前3小节的内容。

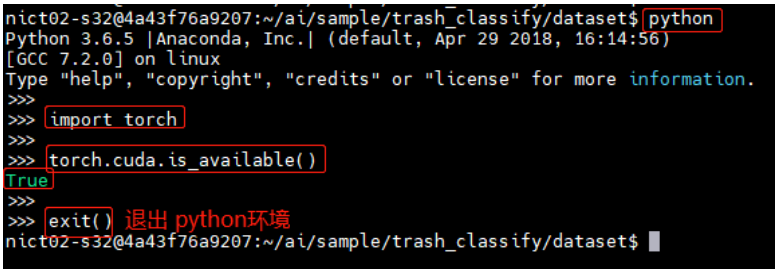
步骤2：数据集上传至训练服务器

- 将制作好的数据集，上传到服务器中，本文提供的垃圾分类案例的数据集已经存放在训练服务器的~/ai/sample/trash_classify/dataset 目录下了。



步骤3：检查cuda环境

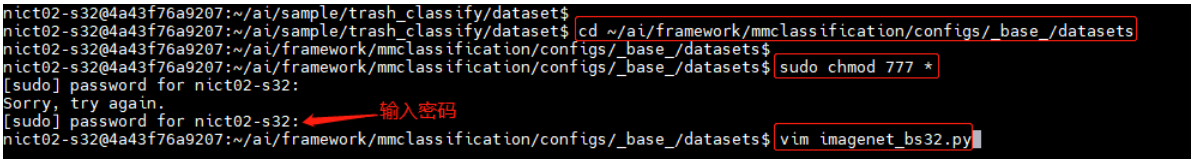
- 查看是否能获取cuda，首先在训练服务器的命令行终端输入 python，然后输入import torch，再输入torch.cuda.is_available()，若显示true 则表示cuda 可获取，否则检查环境是否正确，退出输入exit()即可。



步骤4：修改源码和配置文件

- 在训练服务器的命令行终端，执行下面的命令，修改开源代码，需要修改的文件路径在`~/ai/framework/mmclassification/configs/_base_/datasets`目录下，这里需替换自己的数据集实际路径，按照下图所示进行修改，包括mean、std、resize、crop、data_prefix确认修改，这个务必配置对，否则影响模型训练的效果。

```
cd ~/ai/framework/mmclassification/configs/_base_/datasets

sudo chmod 777 *

vim imagenet_bs32.py
```



```
# dataset settings
dataset_type = 'ImageNet'
img_norm_cfg = dict(
    mean=[127.5, 127.5, 127.5], std=[127.5, 127.5, 127.5], to_rgb=True)
train_pipeline = [
    dict(type='LoadImageFromFile'),
    # dict(type='RandomResizedCrop', size=224),
    dict(type='Resize', size=(256, 256)),
    dict(type='CenterCrop', crop_size=224),
    dict(type='RandomFlip', flip_prob=0.5, direction='horizontal'),
    dict(type='Normalize', **img_norm_cfg),
    dict(type='ImageToTensor', keys=['img']),
    dict(type='ToTensor', keys=['gt_label']),
    dict(type='Collect', keys=['img', 'gt_label'])
]
test_pipeline = [
    dict(type='LoadImageFromFile'),
    dict(type='Resize', size=(256, 256)),
    dict(type='CenterCrop', crop_size=224),
    dict(type='Normalize', **img_norm_cfg),
    dict(type='ImageToTensor', keys=['img']),
    dict(type='Collect', keys=['img'])
]
data = dict(
    samples_per_gpu=32,
    workers_per_gpu=2,
    train=dict(
        type=dataset_type,
        data_prefix='/home/nict02-s32/ai/sample/trash_classify/dataset',
        pipeline=train_pipeline),
    val=dict(
        type=dataset_type,
        data_prefix='/home/nict02-s32/ai/sample/trash_classify/dataset',
        # ann_file='data/imagenet/meta/val.txt',
        pipeline=test_pipeline),
    test=dict(
        # replace `data/val` with `data/test` for standard test
        type=dataset_type,
        data_prefix='/home/nict02-s32/ai/sample/trash_classify/dataset',
        # ann_file='data/imagenet/meta/val.txt',
        pipeline=test_pipeline))
evaluation = dict(interval=1, metric='accuracy')
~
```

需要训练的数据集的绝对路径

- 在训练服务器的命令行终端，执行下面的命令，修改`~/ai/framework/mmclassification/configs/_base_/models`目录下resnet18.py，按下图修改即可：

```
cd  ../models

sudo chmod 777 *

vim resnet18.py
```

```
nict02-s32@4a43f76a9207:~/ai/framework/mmclassification/configs/_base_/datasets$
nict02-s32@4a43f76a9207:~/ai/framework/mmclassification/configs/_base_/datasets$ cd  ../models
nict02-s32@4a43f76a9207:~/ai/framework/mmclassification/configs/_base_/models$
nict02-s32@4a43f76a9207:~/ai/framework/mmclassification/configs/_base_/models$ sudo chmod 777 *
nict02-s32@4a43f76a9207:~/ai/framework/mmclassification/configs/_base_/models$
nict02-s32@4a43f76a9207:~/ai/framework/mmclassification/configs/_base_/models$ vim resnet18.py
nict02-s32@4a43f76a9207:~/ai/framework/mmclassification/configs/_base_/models$
```

```
# model settings
model = dict(
    type='ImageClassifier',
    backbone=dict(
        type='ResNet',
        depth=18,
        num_stages=4,
        out_indices=(3, ),
        style='pytorch'),
    neck=dict(type='GlobalAveragePooling'),
    head=dict(
        type='LinearClsHead',
        num_classes=13,
        in_channels=512,
        loss=dict(type='CrossEntropyLoss', loss_weight=1.0),
        topk=(1, 5),
    ))
```

① num_classes根据实际训练数据集的种类来填写

② 当num_classes<5时，topk的第二个值=num_classes
当num_classes>=5时，topk的第二个值为5

步骤5：查看显卡资源

- 使命令 `nvidia-smi` 查看哪张显卡空闲或 Volatile GPU-Util 利用率较低，假如第0张显卡空闲，我们就进入到下一步骤。

```
$ nvidia-smi
Fri Feb 10 18:51:08 2023
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 515.48.07    Driver Version: 515.48.07    CUDA Version: 11.7     |
|-------------------------------+----------------------+----------------------+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|                               |                      |               MIG M. |
|===============================+======================+======================|
|   0  Tesla T4            Off  | 00000000:86:00.0 Off |                    0 |
| N/A   35C    P0    28W /  70W |   1371MiB / 15360MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
|   1  Tesla T4            Off  | 00000000:87:00.0 Off |                    0 |
| N/A   38C    P0    27W /  70W |      2MiB / 15360MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
|   2  Tesla T4            Off  | 00000000:AF:00.0 Off |                    0 |
| N/A   36C    P0    27W /  70W |      2MiB / 15360MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
|   3  Tesla T4            Off  | 00000000:D8:00.0 Off |                    0 |
| N/A   38C    P0    28W /  70W |      2MiB / 15360MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
```

步骤6：开始训练

- 切换到mmclassification目录下，在训练服务器的命令行终端，执行下面的命令，开始进行模型的训练

```
# 进入到mmclassification目录下
cd ~/ai/framework/mmclassification/

# 开始进行模型训练
CUDA_VISIBLE_DEVICES=0 bash ./tools/dist_train.sh
configs/resnet/resnet18_b32x8_imagenet.py 1 --work-dir ./ckpt

# 对上述命令阐述如下：
# CUDA_VISIBLE_DEVICES=0 表示使用GPU编号为0的显卡进行训练
# dist_train.sh - 训练sh脚本
# configs/resnet/resnet18_b32x8_imagenet.py - 训练依赖的配置
# 1 - GPU个数
# --work-dir ./ckp - 模型存放的路径
```

```
nict02-s32@4a43f76a9207:~/ai/framework/mmclassification/configs/_base_/datasets$
nict02-s32@4a43f76a9207:~/ai/framework/mmclassification/configs/_base_/datasets$ cd ~/ai/framework/mmclassification/
nict02-s32@4a43f76a9207:~/ai/framework/mmclassification$
nict02-s32@4a43f76a9207:~/ai/framework/mmclassification$ CUDA_VISIBLE_DEVICES=0 bash ./tools/dist_train.sh configs/resnet/resnet18_b32x8_imagenet.py 1 --work-dir ./ckpt
/usr/local/anaconda3/lib/python3.6/site-packages/torch/distributed/launch.py:186: FutureWarning: The module torch.distributed.launch is deprecated
and will be removed in future. Use torchrun.
Note that --use_env is set by default in torchrun.
If your script expects `--local_rank` argument to be set, please
change it to read from `os.environ['LOCAL_RANK']` instead. See
https://pytorch.org/docs/stable/distributed.html#launch-utility for
further instructions

  FutureWarning,
fatal: not a git repository (or any parent up to mount point /home)
Stopping at filesystem boundary (GIT_DISCOVERY_ACROSS_FILESYSTEM not set).
2023-02-10 18:50:53,205 - mmcls - INFO - Environment info:
------------------------------------------------------------
```

- 当开始训练之后，我们可以看到显卡0的Volatile GPU-Util 利用率就已经大于0%了，代表该显卡正在使用中。

```
~/nicu2023$ nvidia-smi
Fri Feb 10 18:54:42 2023
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 515.48.07    Driver Version: 515.48.07    CUDA Version: 11.7     |
|-------------------------------+----------------------+----------------------+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|                               |                      |               MIG M. |
|===============================+======================+======================|
|   0  Tesla T4            Off  | 00000000:86:00.0 Off |                    0 |
| N/A   51C    P0    36W /  70W |   2515MiB / 15360MiB |     27%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
|   1  Tesla T4            Off  | 00000000:87:00.0 Off |                    0 |
| N/A   33C    P8    16W /  70W |      2MiB / 15360MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
|   2  Tesla T4            Off  | 00000000:AF:00.0 Off |                    0 |
| N/A   31C    P8    16W /  70W |      2MiB / 15360MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
|   3  Tesla T4            Off  | 00000000:D8:00.0 Off |                    0 |
| N/A   33C    P8    16W /  70W |      2MiB / 15360MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------+
```

- 注:训练时务必指定某张空闲显卡或 Volatile GPU-Util 利用率较低的显卡进行训练。
- 训练成功后，即可在 ~/ai/framework/mmclassification/ckpt 目录下，查看生成的.pth 模型，其中最后一个 epoch_100.pth是我们所需要的。

```
-rw-rw-r--  1 nict02-s32 nict02-s32  86M Feb 10 22:41 epoch_80.pth
-rw-rw-r--  1 nict02-s32 nict02-s32  86M Feb 10 22:44 epoch_81.pth
-rw-rw-r--  1 nict02-s32 nict02-s32  86M Feb 10 22:47 epoch_82.pth
-rw-rw-r--  1 nict02-s32 nict02-s32  86M Feb 10 22:49 epoch_83.pth
-rw-rw-r--  1 nict02-s32 nict02-s32  86M Feb 10 22:52 epoch_84.pth
-rw-rw-r--  1 nict02-s32 nict02-s32  86M Feb 10 22:55 epoch_85.pth
-rw-rw-r--  1 nict02-s32 nict02-s32  86M Feb 10 22:58 epoch_86.pth
-rw-rw-r--  1 nict02-s32 nict02-s32  86M Feb 10 23:01 epoch_87.pth
-rw-rw-r--  1 nict02-s32 nict02-s32  86M Feb 10 23:03 epoch_88.pth
-rw-rw-r--  1 nict02-s32 nict02-s32  86M Feb 10 23:06 epoch_89.pth
-rw-rw-r--  1 nict02-s32 nict02-s32  86M Feb 10 19:23 epoch_9.pth
-rw-rw-r--  1 nict02-s32 nict02-s32  86M Feb 10 23:09 epoch_90.pth
-rw-rw-r--  1 nict02-s32 nict02-s32  86M Feb 10 23:12 epoch_91.pth
-rw-rw-r--  1 nict02-s32 nict02-s32  86M Feb 10 23:14 epoch_92.pth
-rw-rw-r--  1 nict02-s32 nict02-s32  86M Feb 10 23:17 epoch_93.pth
-rw-rw-r--  1 nict02-s32 nict02-s32  86M Feb 10 23:20 epoch_94.pth
-rw-rw-r--  1 nict02-s32 nict02-s32  86M Feb 10 23:23 epoch_95.pth
-rw-rw-r--  1 nict02-s32 nict02-s32  86M Feb 10 23:26 epoch_96.pth
-rw-rw-r--  1 nict02-s32 nict02-s32  86M Feb 10 23:28 epoch_97.pth
-rw-rw-r--  1 nict02-s32 nict02-s32  86M Feb 10 23:31 epoch_98.pth
-rw-rw-r--  1 nict02-s32 nict02-s32  86M Feb 10 23:34 epoch_99.pth
lrwxrwxrwx  1 nict02-s32 nict02-s32   13 Feb 10 23:37 latest.pth -> epoch_100.pth
nict02-s32@4a43f76a9207:~/ai/framework/mmclassification/ckpt$
```

步骤7：模型转换

- 目前 NNIE 配套软件及工具链仅支持以 Caffe1.0 框架，使用其他框架的网络模型需要转化为Caffe框架下的模型，具体转换过程如下。
- 在训练服务器的命令行终端，执行下面的命令，对 ~/pytorch_to_caffe_master/example/resnet_pytorch_2_caffe.py进行修改，主要需要修改的是name，num_classes的种类，以及您pytorch模型的绝对路径，具体修改如下图所示，**注意：因为latest.pth只是epoch_100.pth的一个软连接，我们最后指定epoch_100.pth的绝对路径就好。**

```
cd  ~/ai/pytorch_to_caffe_master

vim example/resnet_pytorch_2_caffe.py
```

```
nict02-s32@4a43f76a9207:~$
nict02-s32@4a43f76a9207:~$
nict02-s32@4a43f76a9207:~$ cd  ~/ai/pytorch_to_caffe_master/
nict02-s32@4a43f76a9207:~/ai/pytorch_to_caffe_master$
nict02-s32@4a43f76a9207:~/ai/pytorch_to_caffe_master$ vim example/resnet_pytorch_2_caffe.py
```

```
import sys
sys.path.insert(0,'.')
import torch
from torch.autograd import Variable
from torchvision.models import resnet
import pytorch_to_caffe

if __name__=='__main__':                    开发者所训练的数据集的种类
    name='resnet18'
    resnet18=resnet.resnet18(num_classes=13)
    checkpoint = torch.load("/home/nict02-s32/ai/framework/mmclassification/ckpt/epoch_100.pth", map_location = 'cpu')
    new_sd = {}                                          训练出来的模型的绝对路径
    for k,v in checkpoint['state_dict'].items():
        if not k.endswith('num_batches_tracked'):
            if k.startswith('backbone'):
                k_new = k.split('backbone.')[1]
            if k.startswith('head'):
                k_new = k.split('head.')[1]
            new_sd[k_new] = v

    resnet18.load_state_dict(new_sd)
    resnet18.eval()
    input=torch.ones([1,3,224,224])
     #input=torch.ones([1,3,224,224])
    pytorch_to_caffe.trans_net(resnet18,input,name)
    pytorch_to_caffe.save_prototxt('{}.prototxt'.format(name))
    pytorch_to_caffe.save_caffemodel('{}.caffemodel'.format(name))
~
~
```

- 在训练服务器的命令行终端，执行下面的命令,进行模型的转换：

```
python example/resnet_pytorch_2_caffe.py
```

```
nict02-s32@4a43f76a9207:~/ai/pytorch_to_caffe_master$
nict02-s32@4a43f76a9207:~/ai/pytorch_to_caffe_master$ python example/resnet_pytorch_2_caffe.py
Starting Transform, This will take a while
139672012504184:blob1 was added to blobs
torch ops name: {ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)
  (layer1): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
    (1): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (layer2): Sequential(
    (0): BasicBlock(
```

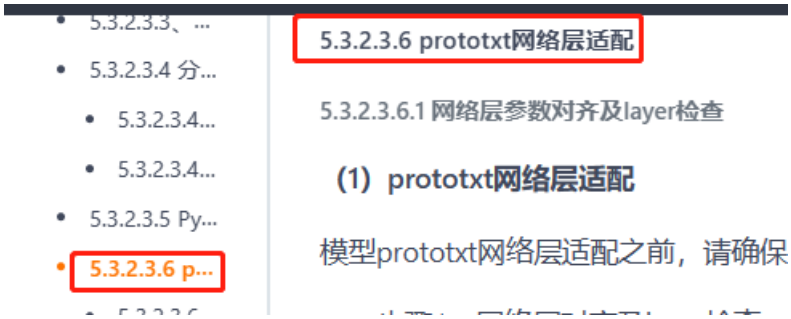
- 转换成功后，即可在pytorch_to_caffe_master目录下生成resnet18.caffemodel和resnet18.prototxt文件。

```
conv20 was added to layers
139672012537272:conv_blob20 was added to blobs
['relu_blob16']
layer4.1.bn2
batch_norm20 was added to layers
139672012537928:batch_norm_blob20 was added to blobs
['conv_blob20']
bn_scale20 was added to layers
['batch_norm_blob20']
add8 was added to layers
139672012538328:add_blob8 was added to blobs
['batch_norm_blob20', 'relu_blob15']
layer4.1.relu
relu17 was added to layers
139672012538248:relu_blob17 was added to blobs
['add_blob8']
avgpool
ave_pool1 was added to layers
139672012538168:ave_pool_blob1 was added to blobs
['relu_blob17']
fc
fc1 was added to layers
139672012538568:fc_blob1 was added to blobs
WARNING: CANNOT FOUND blob 139672012538008
Transform Completed
nict02-s32@4a43f76a9207:~/ai/pytorch_to_caffe_master$ ls
001763.jpg  LICENSE.md  README.md     __pycache__  caffe_analyser.py  funcs.py  pytorch_analyser.py  resnet18.caffemodel  visionmaster
Caffe       Pytorch     __init__.py  analysis     example            model     pytorch_to_caffe.py  resnet18.prototxt
nict02-s32@4a43f76a9207:~/ai/pytorch_to_caffe_master$
```

- 至此，模型转换已经完成。

步骤8：prototxt网络层适配

- 关于prototxt网络层适配的相关操作，可以直接参考社区提供的5.3.2.3.6 prototxt网络层适配 小节的内容即可。



步骤9：模型量化

- 关于模型量化的相关操作，可以直接参考社区提供的 5.3.2.3.7 模型量化 小节的内容即可。

步骤10：模型部署板端和调试

- 关于模型部署板端和调试的相关操作，可以直接参考社区提供的 5.3.2.3.8、模型部署板端和调试 小节的内容即可。

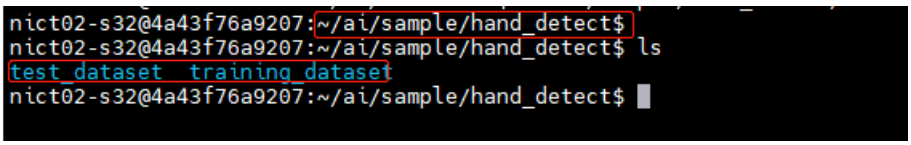
3、检测网模型训练及模型转换

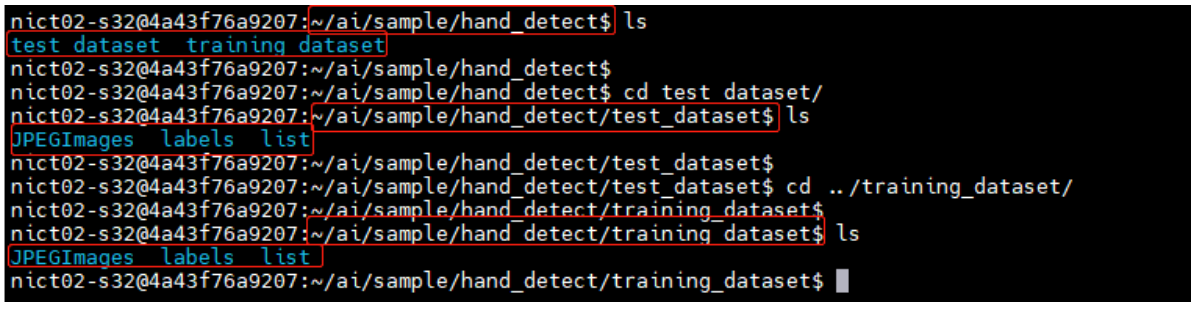
步骤1：数据集准备

- 在进行检测网训练之前，请您仔细阅读社区提供的检测网训练前的准备工作，如：数据集的制作、数据集的清洗、数据集的标注等，具体请参考《5.3.2.4检测网》第1小节的内容。

步骤2：数据集上传至训练服务器

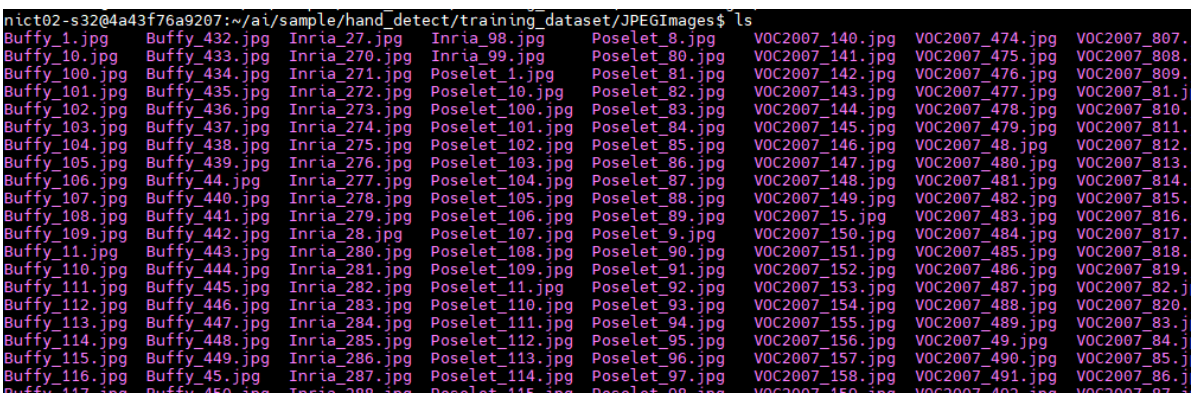
- 将制作好的数据集，上传到服务器中，本文提供的手势检测案例的数据集已经存放在训练服务器的~/ai/sample/hand_detect 目录下了。



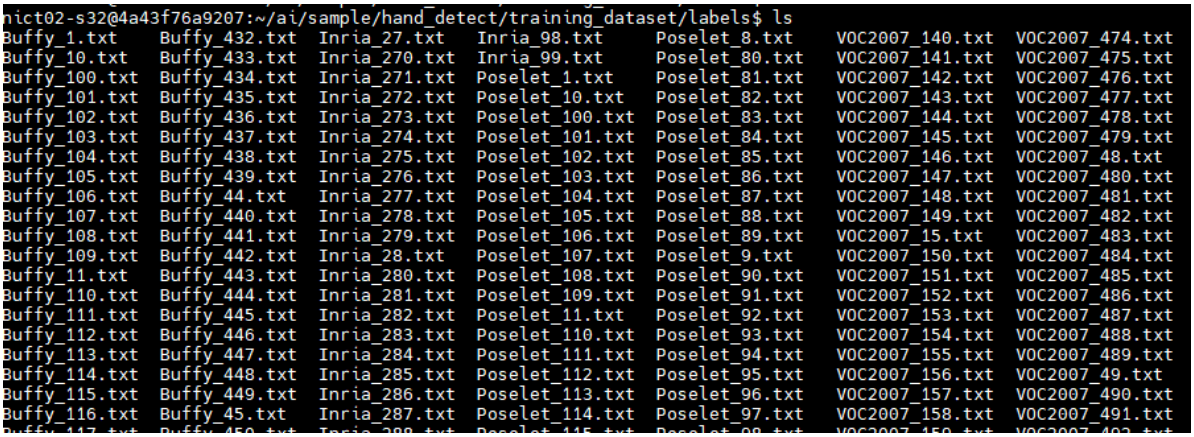



```
# test_dataset为测试数据
# training_dataset为训练数据
# JPEGImages目录下是需要进行训练的数据集图片
# labels目录下的txt是JPEGImages目录下对应图片所标注的labels
# list目录下的hand_train.txt文件中保存的是JPEGImages目录下所有图片的绝对路径。
# 注意：所以的txt文件都必须是linux格式的，可以使用 dos2unix 工具进行文件格式的转换
# 使用方法： dos2unix filename    （如果还不知道如何使用，可上网咨询度娘）
```

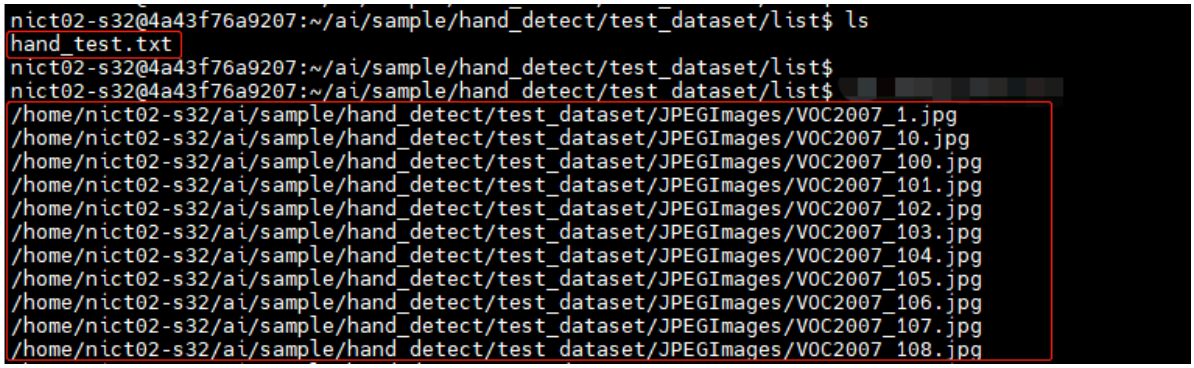
- JPEGImages目录下的图片，就是需要进行训练的数据集图片

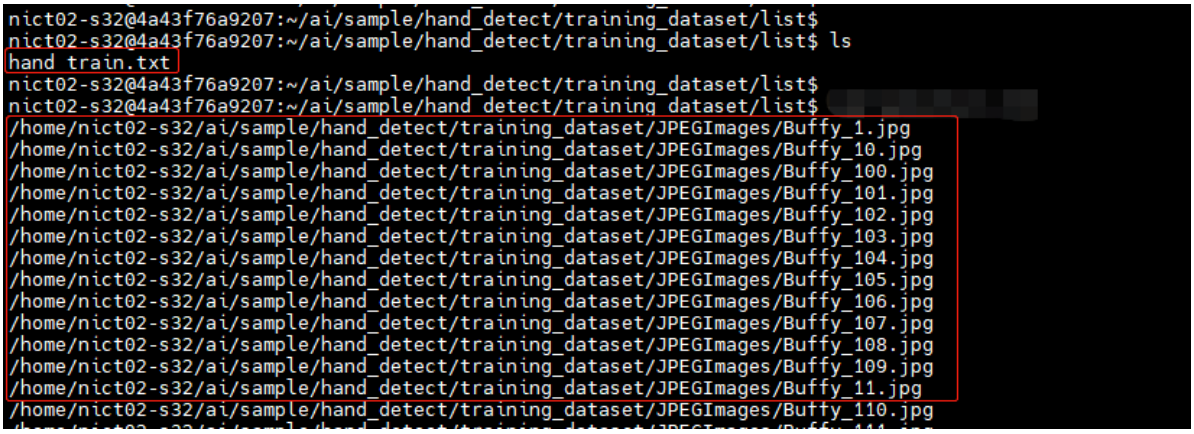


- labels目录下的txt，是JPEGImages目录下对应图片所标注的labels，文件名字都是一模一样的，只是文件后缀有不一样。



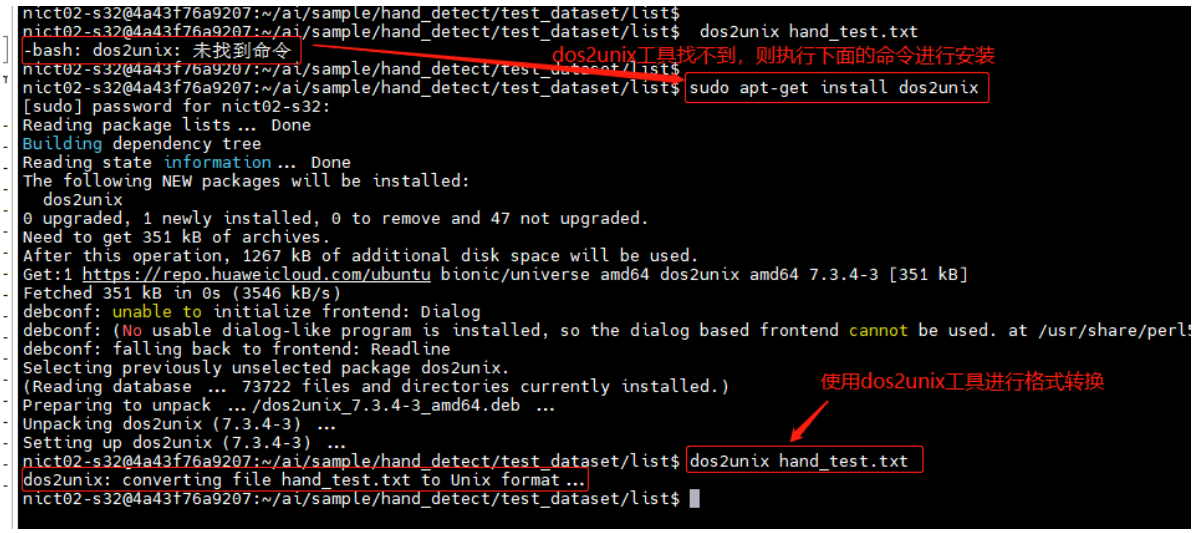
- list目录下的hand_train.txt文件中保存的是JPEGImages目录下所有图片的绝对路径，**注意：如果此处的绝对路径不对，后面模型训练的时候会因为找不到训练的图片，而导致训练失败**，所以请一定要把test_dataset/list/hand_test.txt和training_dataset/list/hand_train.txt中的图片绝对路径修改正确。

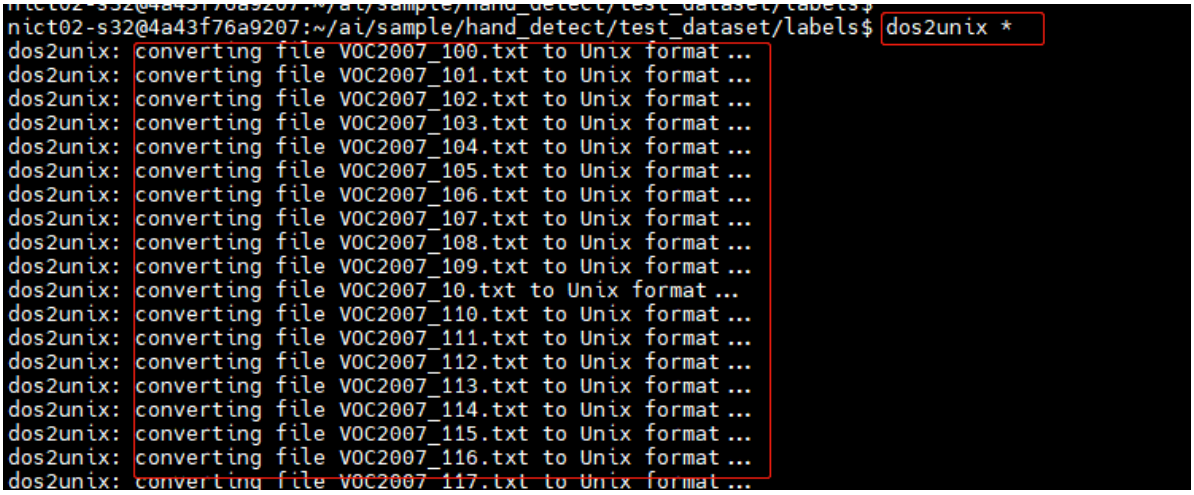




- 注意：所有的txt文件都必须是linux格式的，可以使用 dos2unix 工具进行文件格式的转换，使用方法： dos2unix filename （如果还不知道如何使用，可上网咨询度娘）

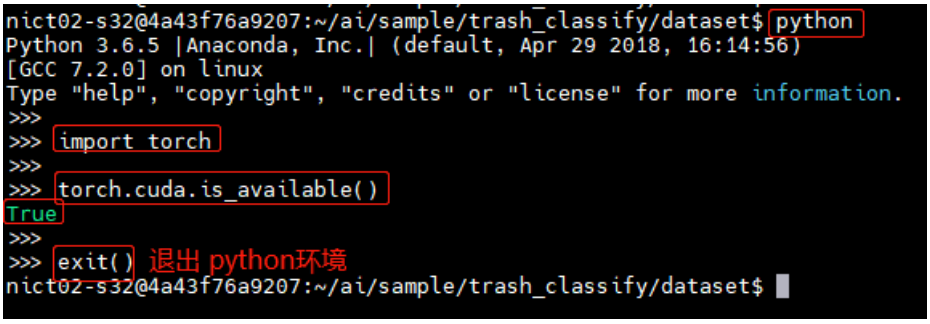
```
sudo apt-get install dos2unix  # 如果服务器没有dos2unix这个软件的话，执行这条命令，先进行安装

dos2unix hand_test.txt         # 修改某个文件的文件格式

dos2unix  *                    # 修改该目录下的所有文件的文件格式
```





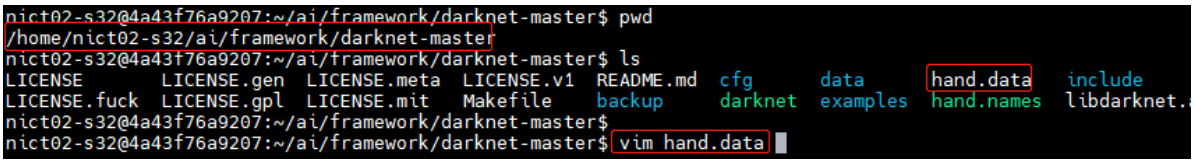
步骤3：检查cuda环境

- 查看是否能获取cuda,首先在训练服务器的命令行终端输入 python,然后输入import torch，再输入torch.cuda.is_available()，若显示true 则表示cuda 可获取，否则检查环境是否正确，退出输入exit()即可



步骤4：修改源码和配置文件

- 修改配置文件hand.data中的参数，注意：hand.data 文件在 **~/ai/framework/darknet-master** 目录下



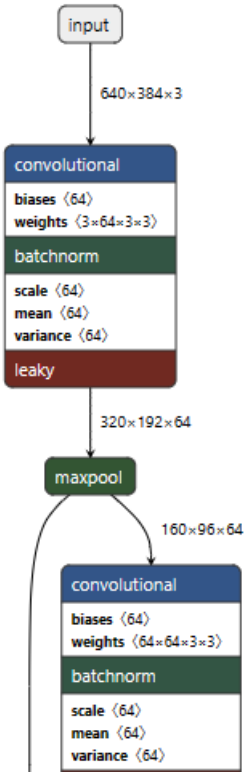
```
classes=1
train =/home/nict02-s32/ai/sample/hand_detect/training_dataset/list/hand_train.txt
valid =/home/nict02-s32/ai/sample/hand_detect/test_dataset/list/hand_test.txt
names =/home/nict02-s32/ai/framework/darknet-master/hand.names
backup =/home/nict02-s32/ai/framework/darknet-master/backup
~
```

```
# classes：为指定类别数量，手部检测模型为1，其他模型需根据实际场景进行填写
#  train：指定训练数据list的路径
#  valid：指定测试数据list的路径
#  names：指定类别的名字
# backup：指定训练后权重的保存路径
```

- 修改~/ai/framework/darknet-master目录下的hand.name，hand.name里面的内容，就是需要检测的类型。本文需要检测的物体只有手，因此hand.name里面的内容就只有hand，如果您需要检测多种物体，可每一行书写一种物体。

```
nict02-s32@4a43f76a9207:~/ai/framework/darknet-master$
nict02-s32@4a43f76a9207:~/ai/framework/darknet-master$ cat hand.names
hand
nict02-s32@4a43f76a9207:~/ai/framework/darknet-master$
```

- 注意： ~/ai/framework/darknet-master/cfg/下的 resnet18.cfg，可以通过 https://netron.app/ 查看网络结构和参数，部分网络结构截图如下图所示：



步骤5： 查看显卡资源

- 使命令 `nvidia-smi` 查看哪张显卡空闲或 Volatile GPU-Util 利用率较低，假如第1张显卡空闲，我们就进入到下一步骤。

```
$ nvidia-smi
Fri Feb 10 18:51:08 2023
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 515.48.07    Driver Version: 515.48.07    CUDA Version: 11.7     |
|-------------------------------+----------------------+----------------------+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|                               |                      |               MIG M. |
|===============================+======================+======================|
|   0  Tesla T4            Off  | 00000000:86:00.0 Off |                    0 |
| N/A   35C    P0    28W /  70W |   1371MiB / 15360MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
|   1  Tesla T4            Off  | 00000000:87:00.0 Off |                    0 |
| N/A   38C    P0    27W /  70W |      2MiB / 15360MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
|   2  Tesla T4            Off  | 00000000:AF:00.0 Off |                    0 |
| N/A   36C    P0    27W /  70W |      2MiB / 15360MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
|   3  Tesla T4            Off  | 00000000:D8:00.0 Off |                    0 |
| N/A   38C    P0    28W /  70W |      2MiB / 15360MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
```

步骤6：开始训练

- 切换到darknet-master目录下，在训练服务器的命令行终端，执行下面的命令，开始进行模型的训练

```
cd ~/ai/framework/darknet-master

mkdir backup

CUDA_VISIBLE_DEVICES=1 ./darknet detector train hand.data cfg/resnet18.cfg

# 对上述命令阐述如下：
# CUDA_VISIBLE_DEVICES=1 表示使用GPU编号为1的显卡进行训练
# darknet为可执行文件
# detector为必选参数，直接沿用即可
# train指定当前模式为训练模式，测试时候需要改成test或者valid。
# hand.data是一个文件，文件中指定了训练中数据、类别、模型存储路径等信息。
# resnet18.cfg是一个文件，文件中指定了训练的模型。
```

```
nict02-s32@4a43f76a9207:~$
nict02-s32@4a43f76a9207:~$ cd ~/ai/framework/darknet-master
nict02-s32@4a43f76a9207:~/ai/framework/darknet-master$
nict02-s32@4a43f76a9207:~/ai/framework/darknet-master$ mkdir backup
nict02-s32@4a43f76a9207:~/ai/framework/darknet-master$
nict02-s32@4a43f76a9207:~/ai/framework/darknet-master$ CUDA_VISIBLE_DEVICES=1 ./darknet detector train hand.data cfg/resnet18.cfg
resnet18
layer     filters    size              input                output
    0 conv     64  3 x 3 / 2   640 x 384 x   3   ->   320 x 192 x  64  0.212 BFLOPs
    1 max          2 x 2 / 2   320 x 192 x  64   ->   160 x  96 x  64
    2 conv     64  3 x 3 / 1   160 x  96 x  64   ->   160 x  96 x  64  1.132 BFLOPs
    3 conv     64  3 x 3 / 1   160 x  96 x  64   ->   160 x  96 x  64  1.132 BFLOPs
    4 res    1                 160 x  96 x  64   ->   160 x  96 x  64
    5 conv     64  3 x 3 / 1   160 x  96 x  64   ->   160 x  96 x  64  1.132 BFLOPs
    6 conv     64  3 x 3 / 1   160 x  96 x  64   ->   160 x  96 x  64  1.132 BFLOPs
    7 res    4                 160 x  96 x  64   ->   160 x  96 x  64
    8 max          2 x 2 / 2   160 x  96 x  64   ->    80 x  48 x  64
    9 conv    128  1 x 1 / 1    80 x  48 x  64   ->    80 x  48 x 128  0.063 BFLOPs
   10 conv    128  3 x 3 / 1    80 x  48 x 128   ->    80 x  48 x 128  1.132 BFLOPs
   11 conv    128  1 x 1 / 1    80 x  48 x 128   ->    80 x  48 x 128  0.126 BFLOPs
   12 res    9                  80 x  48 x 128   ->    80 x  48 x 128
   13 conv    128  1 x 1 / 1    80 x  48 x 128   ->    80 x  48 x 128  0.126 BFLOPs
   14 conv    128  3 x 3 / 1    80 x  48 x 128   ->    80 x  48 x 128  1.132 BFLOPs
   15 conv    128  1 x 1 / 1    80 x  48 x 128   ->    80 x  48 x 128  0.126 BFLOPs
   16 res   13                  80 x  48 x 128   ->    80 x  48 x 128
   17 max          2 x 2 / 2    80 x  48 x 128   ->    40 x  24 x 128
   18 conv    256  1 x 1 / 1    40 x  24 x 128   ->    40 x  24 x 256  0.063 BFLOPs
   19 conv    256  3 x 3 / 1    40 x  24 x 256   ->    40 x  24 x 256  1.132 BFLOPs
```

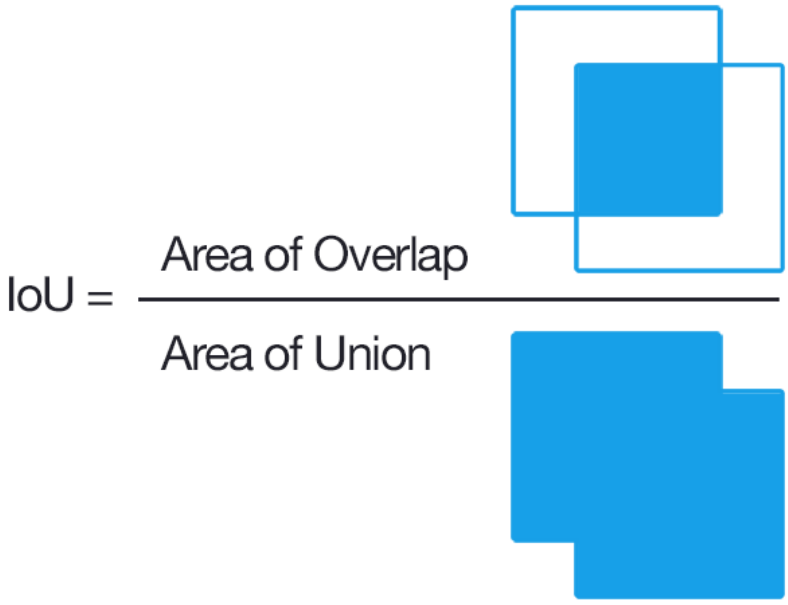
- 训练过程如下图所示：

```
Loaded: 0.000038 seconds
Region Avg IOU: 0.886993, Class: 1.000000, Obj: 0.858286, No Obj: 0.002914, Avg Recall: 1.000000,  count: 12
Region Avg IOU: 0.832719, Class: 0.999996, Obj: 0.854385, No Obj: 0.002407, Avg Recall: 1.000000,  count: 10
Region Avg IOU: 0.828295, Class: 1.000000, Obj: 0.823431, No Obj: 0.003020, Avg Recall: 1.000000,  count: 14
Region Avg IOU: 0.767622, Class: 0.999999, Obj: 0.747484, No Obj: 0.001963, Avg Recall: 1.000000,  count: 9
Region Avg IOU: 0.783933, Class: 1.000000, Obj: 0.679804, No Obj: 0.002955, Avg Recall: 1.000000,  count: 14
Region Avg IOU: 0.821345, Class: 0.999999, Obj: 0.825026, No Obj: 0.002427, Avg Recall: 1.000000,  count: 8
Region Avg IOU: 0.823367, Class: 0.923077, Obj: 0.831396, No Obj: 0.002912, Avg Recall: 1.000000,  count: 13
Region Avg IOU: 0.763027, Class: 0.999998, Obj: 0.745521, No Obj: 0.003229, Avg Recall: 0.937500,  count: 16
165648: 1.626298, 1.757994 avg, 0.001000 rate, 0.348999 seconds, 7951104 images
Loaded: 0.452634 seconds
Region Avg IOU: 0.808926, Class: 1.000000, Obj: 0.687957, No Obj: 0.002875, Avg Recall: 1.000000,  count: 17
Region Avg IOU: 0.806398, Class: 1.000000, Obj: 0.748294, No Obj: 0.003240, Avg Recall: 1.000000,  count: 15
Region Avg IOU: 0.666447, Class: 0.937500, Obj: 0.678420, No Obj: 0.002468, Avg Recall: 0.812500,  count: 16
Region Avg IOU: 0.795317, Class: 0.999999, Obj: 0.755479, No Obj: 0.002694, Avg Recall: 0.846154,  count: 13
Region Avg IOU: 0.646040, Class: 0.999997, Obj: 0.585359, No Obj: 0.003021, Avg Recall: 0.708333,  count: 24
Region Avg IOU: 0.771670, Class: 0.999845, Obj: 0.727320, No Obj: 0.002294, Avg Recall: 1.000000,  count: 11
Region Avg IOU: 0.861634, Class: 1.000000, Obj: 0.851164, No Obj: 0.003151, Avg Recall: 1.000000,  count: 13
Region Avg IOU: 0.759751, Class: 1.000000, Obj: 0.729890, No Obj: 0.003590, Avg Recall: 0.950000,  count: 20
165649: 2.393668, 1.821562 avg, 0.001000 rate, 0.345652 seconds, 7951152 images
Loaded: 0.306501 seconds
Region Avg IOU: 0.733155, Class: 1.000000, Obj: 0.751796, No Obj: 0.003858, Avg Recall: 0.882353,  count: 17
Region Avg IOU: 0.602319, Class: 0.928571, Obj: 0.637364, No Obj: 0.002698, Avg Recall: 0.642857,  count: 14
Region Avg IOU: 0.842528, Class: 0.937500, Obj: 0.852856, No Obj: 0.003894, Avg Recall: 1.000000,  count: 16
Region Avg IOU: 0.702595, Class: 0.888887, Obj: 0.663856, No Obj: 0.003068, Avg Recall: 0.777778,  count: 18
Region Avg IOU: 0.592232, Class: 0.849999, Obj: 0.551057, No Obj: 0.002706, Avg Recall: 0.750000,  count: 20
Region Avg IOU: 0.818521, Class: 1.000000, Obj: 0.752892, No Obj: 0.002339, Avg Recall: 1.000000,  count: 9
Region Avg IOU: 0.739972, Class: 1.000000, Obj: 0.744841, No Obj: 0.001861, Avg Recall: 1.000000,  count: 9
Region Avg IOU: 0.822828, Class: 0.999999, Obj: 0.798817, No Obj: 0.003436, Avg Recall: 1.000000,  count: 16
165650: 1.803199, 1.819725 avg, 0.001000 rate, 0.343814 seconds, 7951200 images
Loaded: 0.224971 seconds
Region Avg IOU: 0.767470, Class: 0.999999, Obj: 0.764907, No Obj: 0.003134, Avg Recall: 0.928571,  count: 14
Region Avg IOU: 0.775499, Class: 0.923077, Obj: 0.761286, No Obj: 0.003453, Avg Recall: 0.846154,  count: 13
Region Avg IOU: 0.747874, Class: 0.923077, Obj: 0.709428, No Obj: 0.002432, Avg Recall: 0.923077,  count: 13
Region Avg IOU: 0.511091, Class: 0.882353, Obj: 0.550108, No Obj: 0.002715, Avg Recall: 0.588235,  count: 17
Region Avg IOU: 0.775522, Class: 1.000000, Obj: 0.751023, No Obj: 0.002379, Avg Recall: 1.000000,  count: 12
Region Avg IOU: 0.695097, Class: 0.941176, Obj: 0.708488, No Obj: 0.002882, Avg Recall: 0.823529,  count: 17
Region Avg IOU: 0.709576, Class: 0.928571, Obj: 0.717968, No Obj: 0.002610, Avg Recall: 0.857143,  count: 14
Region Avg IOU: 0.838552, Class: 1.000000, Obj: 0.789735, No Obj: 0.004420, Avg Recall: 1.000000,  count: 18
165651: 2.080095, 1.845762 avg, 0.001000 rate, 0.325313 seconds, 7951248 images
Loaded: 0.022999 seconds
```

- 当开始训练之后，我们可以看到显卡0的Volatile GPU-Util 利用率就已经大于0%了，代表该显卡正在使用中。

```
nict02-s32@4a43f76a9207:~$ nvidia-smi
Mon Feb 13 16:33:10 2023
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 515.48.07    Driver Version: 515.48.07    CUDA Version: 11.7     |
|-------------------------------+----------------------+----------------------+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|                               |                      |               MIG M. |
|===============================+======================+======================|
|   0  Tesla T4            Off  | 00000000:86:00.0 Off |                    0 |
| N/A   32C    P0    28W /  70W |      2MiB / 15360MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
|   1  Tesla T4            Off  | 00000000:87:00.0 Off |                    0 |
| N/A   58C    P0    70W /  70W |   2243MiB / 15360MiB |     93%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
|   2  Tesla T4            Off  | 00000000:AF:00.0 Off |                    0 |
| N/A   33C    P0    27W /  70W |      2MiB / 15360MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
|   3  Tesla T4            Off  | 00000000:D8:00.0 Off |                    0 |
| N/A   35C    P0    27W /  70W |      2MiB / 15360MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------+
| Processes:                                                                  |
|  GPU   GI   CI        PID   Type   Process name                  GPU Memory |
|        ID   ID                                                   Usage      |
|=============================================================================|
```

- 注:训练时务必指定某张空闲显卡或 Volatile GPU-Util 利用率较低的显卡进行训练。
- 对终端输出的一个截图做如下解释：
    - Region Avg IOU：表示当前的subdivision内图片的平均IOU，代表预测的矩形框和真实目标的交集与并集之比，若为100%，表示我们已经拥有了完美的检测，即我们的矩形框跟目标完美重合，若为其他值较小值，表示这个模型需要进一步训练。



- Class：标注物体分类的正确率，期望该值趋近于1。
- Obj：越接近1越好。
- No Obj：期望该值越来越小，但不为零。
- Avg Recall：在recall/count中定义的，是当前模型在所有subdivision图片中检测出的正样本与实际的正样本的比值。
- count：count后的值是所有的当前subdivision图片中包含正样本的图片的数量。

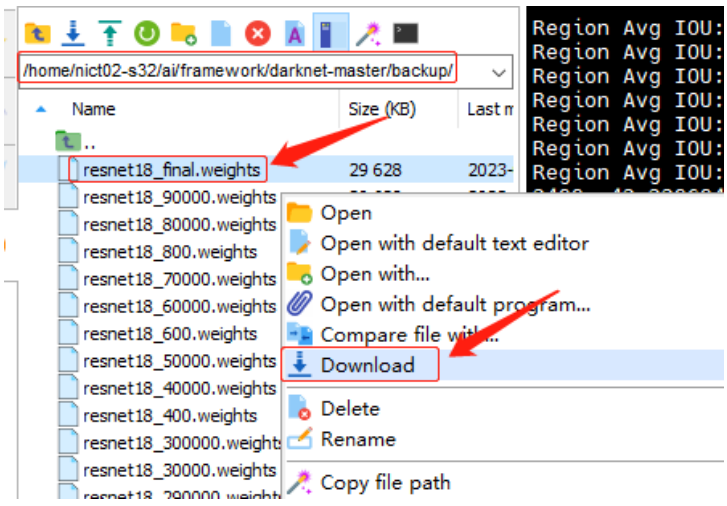
由于检测网训练时间较长，请耐心等待。

- 训练成功后，即可在~/ai/framework/darknet-master/backup 目录下查看.weights文件，若出现resnet18_final.weights表示最终训练完成，如下图所示：

```
resnet18.backup          resnet18_140000.weights  resnet18_20000.weights   resnet18_260000.weights  resnet18_400.weights     resnet18_70000.weights
resnet18_100.weights     resnet18_150000.weights  resnet18_200000.weights  resnet18_270000.weights  resnet18_40000.weights   resnet18_800.weights
resnet18_10000.weights   resnet18_160000.weights  resnet18_210000.weights  resnet18_280000.weights  resnet18_500.weights     resnet18_80000.weights
resnet18_100000.weights  resnet18_170000.weights  resnet18_220000.weights  resnet18_290000.weights  resnet18_50000.weights   resnet18_900.weights
resnet18_110000.weights  resnet18_180000.weights  resnet18_230000.weights  resnet18_300.weights     resnet18_600.weights     resnet18_90000.weights
resnet18_120000.weights  resnet18_190000.weights  resnet18_240000.weights  resnet18_30000.weights   resnet18_60000.weights   resnet18_final.weights
resnet18_130000.weights  resnet18_200.weights     resnet18_250000.weights  resnet18_300000.weights  resnet18_700.weights
```

步骤7：模型转换

- 从训练服务器的~/ai/framework/darknet-master/backup 目录下，下载训练好的模型文件resnet18_final.weights，到windows终端。



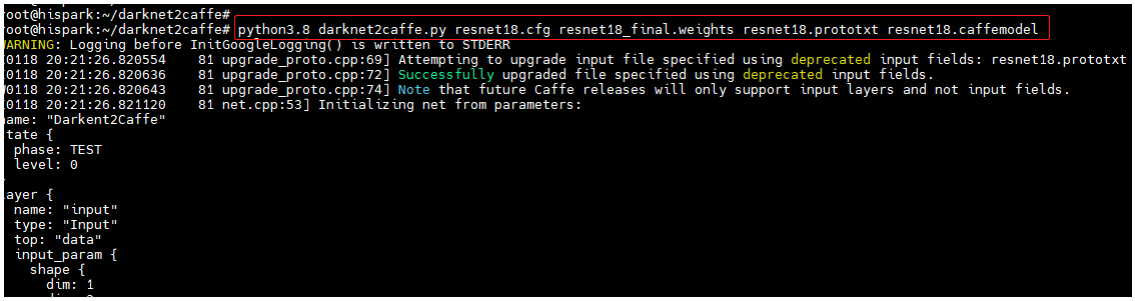
- 步骤2：将刚下载到windows的resnet18_final.weights模型文件，通过samba映射的方式上传到你Ubuntu docker的/root/darknet2caffe/目录下

```
root@hispark:~/darknet2caffe# ls
README.md  __pycache__  caffe_layers  cfg  cfg.py  darknet2caffe.py  prototxt  prototxt.py  resnet18.cfg  resnet18_final.weights  weights
root@hispark:~/darknet2caffe#
```

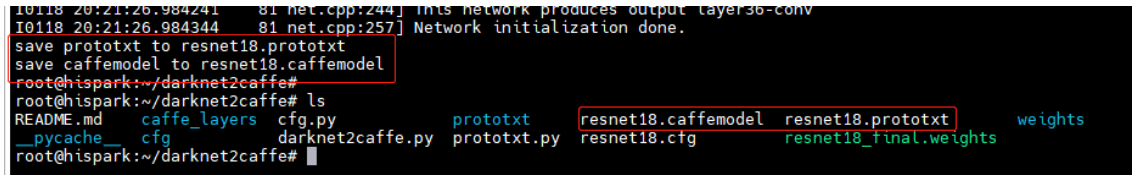
- 步骤3：执行下面的命令，将darknet模型转换为caffe模型。

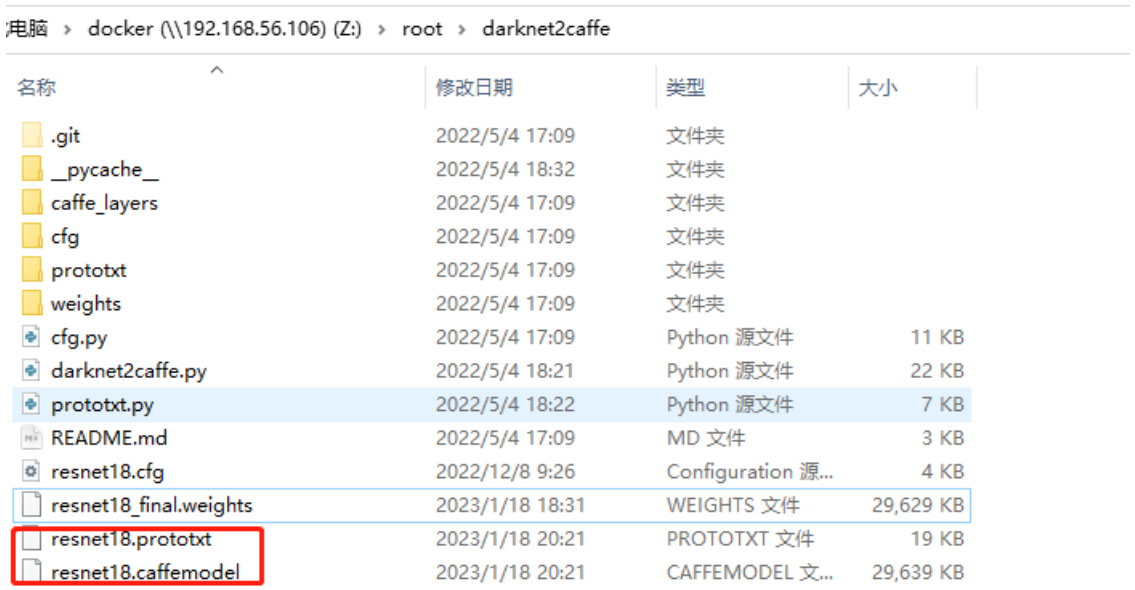
```
# 转换命令遵循：
# python cfg[in] weights[in] prototxt[out] caffemodel[out]
# 本文转换命令如下：
python3.8 darknet2caffe.py resnet18.cfg resnet18_final.weights
resnet18.prototxt resnet18.caffemodel
```

如下图所示：



- 当转换成功后，会在darknet2caffe目录生成一个resnet18.caffemodel和一个resnet18.prototxt文件，如下图所示。





步骤8：模型量化

- 关于模型量化的相关操作，可以直接参考社区提供的 5.3.2.4.4 模型量化 小节的内容即可。

步骤9：模型部署板端和调试

- 关于模型量化的相关操作，可以直接参考社区提供的 5.3.2.4.5、模型部署板端和调试 小节的内容即可。